

DATA 2.0.1: From a Target Formula to a Verified Minimum-Cost Proof

Using the Universal Proof-Horizon Operator as an Exhaustive Backbone
with Heuristic Discovery, Native Verification, and Optional SAT/Algebraic
Methods

Brian Tenneson

M.A. Mathematics; M.S. Applied Data Science - Currently Unaffiliated

btenneson2301@baypath.edu

Version 2.0.1 - Research Draft

August 9, 2026

Scope. This draft proposes and formalizes an automated-theorem-proving architecture for known-theorem benchmarks. It does not claim that the proposed implementation is faster than established ATPs, that arbitrary shortest proofs are computationally feasible, or that undecidable theories become decidable. “Minimum-cost” is always relative to a fixed formal presentation and an effectively proper computable proof-cost measure. A literal fewest-step guarantee requires additional finite-branching hypotheses.

Research provenance. The architecture grows directly from the author’s *Universal Proof-Horizon Operator* paper and from the author’s DATA-ATP/Predator research program. OpenAI ChatGPT was used as an interactive research and document-production interface for formalization, literature comparison, architecture criticism, diagrams, and LaTeX preparation. The author supplied and directed the underlying research questions, including the hybrid use of a Proof-Horizon certifier, a faster heuristic discovery wing, native proof reconstruction, optional SAT and τ -polynomial methods, and the separation between “proof found” and “minimum-cost proof certified.” The verifier, not the generative system, is the authority for certificate correctness.

Copyright. Copyright © Brian Tenneson. All Rights Reserved.

Abstract

Starting from a target theorem φ in a fixed effective formal system, this paper develops DATA 2.0, a hybrid automated-theorem-proving architecture whose output is a native verified proof $\Pi = \langle \psi_1, \dots, \psi_n \rangle$ with $\psi_n = \varphi$. The mathematical backbone is the Universal Proof-Horizon construction: under a total computable proof verifier and an effectively proper computable proof-cost measure, exhaustive search in nondecreasing cost eventually returns a minimum-cost proof whenever the target is a theorem. DATA 2.0 places that exhaustive certifier beside a faster discovery wing containing goal-directed, best-first, portfolio, learned, or other heuristic search policies. Any verified proof found by the fast wing supplies an incumbent upper bound B on the optimum; the Proof-Horizon wing then needs only to exhaust all candidate proofs of cost below B to certify optimality. This creates two distinct success states: *verified proof found* and *minimum-cost verified proof certified*. For finite-branching unit-cost proof-state graphs, ordinary breadth-first search recovers a fewest-step proof and a predecessor map reconstructs the proof vertically. Optional methods convert suitable first-order refutation problems into cumulative SAT approximations, and suitable propositional subproblems into the author's τ -polynomial representation over $\mathbb{F}_2$; neither method is required for the native correctness path. The architecture is designed first for known theorems in a frozen database such as `set.mm`, with conjecture settlement and independence reserved for later extension. The paper gives end-to-end algorithms, diagrams, implementation modules, benchmark quantities, limitations, and open research questions, while deliberately making no performance claim before implementation and testing.

Contents

1	Motivation and mathematical viewpoint	3
2	Symbols and standing conventions	4
3	Effective proof presentations and the target theorem	4
4	The native proof object and vertical reconstruction	5
5	The Proof-Horizon cost layers	6
6	Two-wing DATA 2.0 architecture	7
6.1	Wing 1: fast discovery	7
6.2	Wing 2: Proof-Horizon certifier	7
7	The incumbent bound and minimum-cost certification	8
7.1	Two success states	9
8	SAT/UNSAT as an optional refutation method	9
9	The τ-polynomial method	10
10	Proof horizons, hitting quantities, and progress measures	11
11	Proof-state graphs and the finite-branching shortest-step case	11

12 Generalization beyond <code>set.mm</code> and beyond first-order logic	12
13 Comparison with proof-search and certificate literature	12
14 Implementation plan: a first executable DATA 2.0	13
14.1 Phase 0 modules	13
14.2 Reference pseudocode	14
15 Benchmark illustrations, not performance claims	15
16 Open mathematical and engineering questions	15
16.1 Question 1: which native proof cost should be primary?	16
16.2 Question 2: when does <code>set.mm</code> admit a useful finite-branching shortest-step graph? . . .	16
16.3 Question 3: how much does an incumbent accelerate certification in practice?	16
16.4 Question 4: how should heuristic lemmas interact with the Horizon wing?	16
16.5 Question 5: when is the SAT method useful?	16
16.6 Question 6: when is the τ -algebraic method useful?	16
16.7 Question 7: can proof-state dynamics supply useful progress coordinates?	16
16.8 Question 8: how should known-theorem mode become conjecture mode?	17
17 Conclusion	17
References	17

1 Motivation and mathematical viewpoint

The input to an automated theorem prover is usually written as a target formula, but the desired output is not merely the word “proved.” It is a finite certificate that can be checked independently. In the present paper the target is denoted φ , and the intended output is a native proof

$$\Pi = \langle \psi_1, \psi_2, \dots, \psi_n \rangle, \quad \psi_n = \varphi, \quad (1)$$

where each proof step is licensed by the selected formal system and depends only on allowed earlier material.

The central architectural question is therefore:

How can a machine search aggressively for a proof while preserving a mathematically explicit route to eventual minimum-cost certification?

The answer proposed here separates two roles that are often conflated. A *discovery policy* tries to reach a proof quickly. A *certificate policy* establishes what has actually been proved and, if requested, whether a cheaper proof has been excluded. DATA 2.0 makes the separation physical: a fast discovery wing runs alongside a Proof-Horizon certifier. The two wings share verified discoveries but not logical authority.

This distinction is motivated directly by the author’s Universal Proof-Horizon paper [1]. That paper formalizes the classical computability fact that, for an effective proof system equipped with a suitable proof-cost measure, minimum-cost proof recovery is partial computable on theorem inputs. The paper also distinguishes this universal result from the stronger fewest-line or fewest-step claim, which requires effectively finite branching when ordinary breadth-first search is used. In the present architecture those results become executable design constraints rather than merely metamathematical observations.

Notebook, project, and AI provenance

The immediate architecture was developed by connecting the author’s Proof-Horizon program to the author’s DATA-ATP/Predator work on machine-guided proof search. The design choice is to use the Proof-Horizon construction as an exhaustive mathematical baseline and to let Data-like heuristic agents search for good proofs sooner. ChatGPT assisted in turning that connection into definitions, diagrams, implementation modules, and this manuscript. Correctness is delegated to an independent proof verifier.

Prior art and originality boundary

Textbook/classical material. Exhaustive enumeration of effectively checkable proof objects, breadth-first search on finite-branching graphs, shortest paths under unit edge cost, branch-and-bound ideas, refutation by contradiction, propositional SAT, and independent certificate checking are standard and are not claimed as original. The author’s Proof-Horizon paper itself explicitly states that its core computability result is classical in substance.

Parallel ATP practice. Modern ATP systems already use sophisticated clause selection, saturation, portfolios, learned guidance, SAT/SMT subsolvers, proof logging, and independent checking. Metamath is expressly designed around small, transparent proof verification, while SAT proof formats such as DRAT separate a proof-producing solver from a checker. Instantiation-based first-order provers such as iProver combine first-order instantiation with propositional reasoning. These are important precedents, not components invented here.

Author-developed synthesis proposed in this draft. The contribution claimed here is narrower: use the author’s Proof-Horizon operator as a dedicated *minimum-cost certification wing* in a two-wing ATP; let any verified proof from a faster wing become a shared incumbent bound; continue exhaustive search only below the incumbent cost; reconstruct a native vertical proof object; and report “verified proof found” separately from “minimum-cost proof certified.” This exact engineering synthesis is presented as a research proposal. No worldwide-priority claim is made without a deeper architecture-level prior-art search.

2 Symbols and standing conventions

Symbol	Meaning
T	Fixed effective formal theory or proof presentation.
Γ	Allowed prior library: axioms and/or earlier verified theorems available to the benchmark.
φ	Target theorem to be proved.
Π	Candidate proof object.
ψ_i	The i th displayed derived formula or proof line in a vertical logical presentation.
$V(p, q)$	Total computable verifier; 1 exactly when proof code p proves target code q .
$c(p)$	Computable proof cost.
E_b	Exact cost layer $\{p : c(p) = b\}$.
c^*	Minimum proof cost of φ , when φ is a theorem.
B	Cost of the best currently verified incumbent proof.
S_i	Search/proof state used internally by an ATP engine.
G_n	Optional cumulative propositional grounding through a finite stage n in an optional first-order SAT method.
N^*	First optional grounding stage at which G_n is UNSAT; not the same as proof length.

A benchmark is always defined relative to a frozen formal-system snapshot. If φ is already present in a database such as `set.mm`, the stored proof of φ and any direct target leakage must be masked from the search engine. The independent verifier may of course know the formal rules; what is excluded is the answer the benchmark is meant to rediscover.

3 Effective proof presentations and the target theorem

The formal backbone follows the notation used in the Universal Proof-Horizon paper.

Definition 3.1 (Effective proof presentation). *An effective proof presentation is a triple (T, V, c) such that:*

- (i) T is a formal theory in a recursively presented language;
- (ii) $V(p, q)$ is a total computable verifier returning 1 exactly when p codes a valid T -proof whose conclusion has code q ;
- (iii) $c(p) \in \mathbb{N}$ is a total computable proof cost; and
- (iv) for every $b \in \mathbb{N}$, the sublevel set $\{p : c(p) \leq b\}$ is finite and effectively listable.

The fourth condition is called effective properness.

Natural cost measures include encoded bit length, total symbol count, proof-file byte length, or another representation-level quantity whose bounded sublevel sets are finite and listable. The reason for this condition is operational: “search every cheaper proof first” must be a finite instruction at each finite bound.

Definition 3.2 (Known-theorem benchmark). *A DATA 2.0 known-theorem benchmark is a tuple*

$$\mathcal{B} = (T, \Gamma, \varphi, V, c, H, R),$$

where H records the hardware/resource policy and R records contamination rules such as target-proof masking. The benchmark is successful only when an independently checkable native certificate of φ is produced.

For a Metamath benchmark, the preferred success artifact is a valid Metamath proof using only the allowed database prefix or other explicitly permitted library. The native verifier remains outside the heuristic search policy.

4 The native proof object and vertical reconstruction

The user-facing proof object is written as in (1). In a conventional line-oriented calculus, each ψ_i is either an allowed premise or the consequence of an admissible rule applied to earlier lines. In a stack-oriented or compressed formal language such as Metamath, the native file representation need not literally be a list of formulas in this format. DATA 2.0 therefore distinguishes the *native certificate* from a reconstructed *vertical logical presentation*.

Definition 4.1 (Vertical proof presentation). *A vertical proof presentation for target φ is a finite sequence*

$$\Pi^{\text{vert}} = \langle \psi_1, \dots, \psi_n \rangle, \quad \psi_n = \varphi,$$

together with, for each non-premise line, a record of the rule/assertion, substitution, hypotheses or dependencies, and predecessor information sufficient to justify that line in the selected formal presentation.

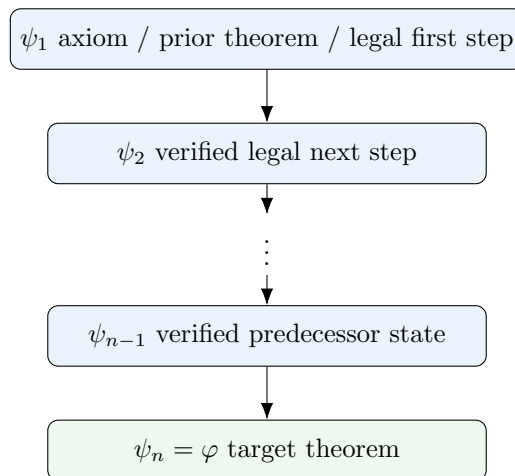


Figure 1: The vertical proof object DATA 2.0 is intended to recover for inspection. The native machine certificate may use a more compact representation, but every displayed line must be traceable to a legal inference.

Proposition 4.2 (Predecessor reconstruction). *Let a proof-state search record, for every newly accepted state, a predecessor state and the legal inference label that produced it. If a target state deriving φ is reached, backtracking these records reconstructs a finite proof path from an initial permitted state to the target.*

Proof. Repeated predecessor lookup strictly follows the finite chain by which the target state was generated. Reversing the resulting finite list gives the sequence of legal transitions from an initial state to the target. The associated inference labels provide the step justifications. $\square$

This proposition is elementary, but architecturally important: the proof certificate is not an afterthought attached to search. The predecessor relation is part of the search state.

5 The Proof-Horizon cost layers

The next result is the mathematical backbone imported from the author’s Universal Proof-Horizon paper [1].

Theorem 5.1 (Universal Proof-Horizon operator, restated). *For every effective proof presentation (T, V, c) and every effective enumeration $\sigma_0, \sigma_1, \dots$ of target sentences, there is a partial computable operator that, on input n , halts exactly when $T \vdash \sigma_n$ and then returns a canonical proof of σ_n having minimum c -cost.*

Proof outline. For $b = 0, 1, 2, \dots$, effectively list the finite exact-cost layer

$$E_b = \{p : c(p) = b\}.$$

Check each candidate using the total verifier. If the target is a theorem, at least one proof exists and its set of costs has a least element b_0 . Every layer below b_0 is finitely exhausted; a valid proof is eventually found in E_{b_0} . No lower-cost proof exists, and a fixed tie-break rule can select a canonical candidate among proofs of equal minimum cost. $\square$

Corollary 5.2 (Minimum proof cost). *For theorem inputs, the minimum proof cost*

$$c^*(\varphi) = \min\{c(p) : p \text{ is a valid } T\text{-proof of } \varphi\}$$

is partial computable as a function of the effectively indexed target.

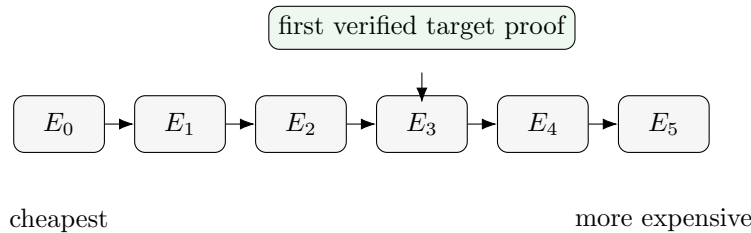


Figure 2: Proof-Horizon search scans effectively finite cost layers in nondecreasing order. The first target proof is minimum-cost relative to the fixed presentation and cost function.

Caution 5.3 (Fewest lines is a stronger claim). *If cost means encoded proof size or another effectively proper representation measure, the theorem applies directly. If cost means literal number of proof lines or graph edges, a bounded line-count layer need not automatically be finite. A fewest-step breadth-first guarantee requires an effectively finite-branching proof-state graph or another condition making each finite depth layer effectively finite.*

For a frozen Metamath library, one possible implementation objective is to engineer the legal move generator so that the selected benchmark graph is effectively finite-branching. If this can be stated and verified, a direct breadth-first “shortest-step” mode becomes available. If not, DATA 2.0 should use an effectively proper encoded-size measure for its universal certifier and report line count only as a secondary statistic.

6 Two-wing DATA 2.0 architecture

DATA 2.0 runs two logically distinct search wings.

6.1 Wing 1: fast discovery

The fast wing is allowed to be opportunistic. Candidate engines include:

- native best-first, beam, A*-like, backward, forward, or bidirectional search;
- Data-ATP-style portfolio agents with different scoring policies;
- shared verified lemma pools and memoized subproofs;
- learned ranking of assertions, substitutions, or proof states;
- restart, checkpoint, and breadcrumb strategies;
- optional first-order SAT/UNSAT and algebraic methods described later.

No Hilbert space-filling curve is required in the baseline architecture. Geometry is a search policy that can be tested later, not a logical dependency.

6.2 Wing 2: Proof-Horizon certifier

The certifier searches completely in nondecreasing c -cost. In a benchmark satisfying the finite-branching unit-step hypotheses, it may instead use ordinary breadth-first search by proof depth. Its purpose is not necessarily to win the race to the first proof. Its purpose is to preserve the eventual optimality guarantee.

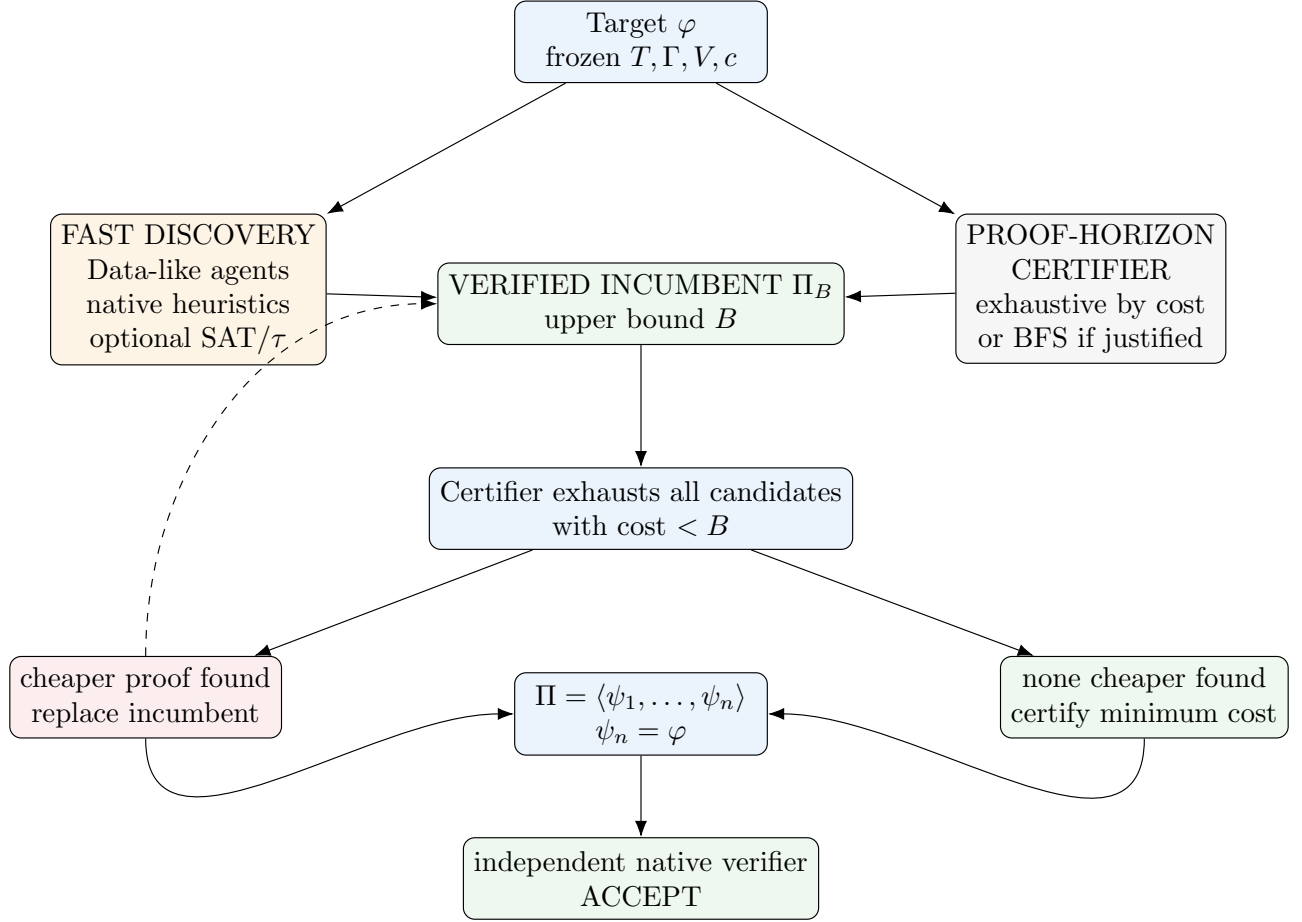


Figure 3: Main DATA 2.0 architecture. Fast discovery supplies good proofs and upper bounds; the Proof-Horizon wing supplies exhaustive certification below the incumbent bound.

Proposition 6.1 (Verifier-gated soundness). *Assume V returns 1 exactly for valid native proofs of the encoded target. If DATA 2.0 reports PROVED only after $V(p, \ulcorner \varphi \urcorner) = 1$, then arbitrary heuristic errors in the fast wing cannot by themselves cause a false proof report.*

Proof. The report condition is the verifier condition. By hypothesis, a candidate passes that condition exactly when it is a valid proof of the target. Heuristic ranking affects which candidates are proposed and when, not the semantics of acceptance. $\square$

This proposition is the reason for keeping the verifier outside the creative or learned search machinery.

7 The incumbent bound and minimum-cost certification

Suppose either wing finds a valid native proof Π_B and the independent verifier accepts it. Define

$$B = c(\Pi_B).$$

Then B is immediately an upper bound on the unknown optimum c^* :

$$c^* \leq B. \tag{2}$$

Theorem 7.1 (Incumbent certification). *Let (T, V, c) be an effective proof presentation and let Π_B be a verified proof of φ with cost B . If every proof candidate of cost strictly less than B has been effectively exhausted and none verifies as a proof of φ , then Π_B has minimum c -cost.*

Proof. Because Π_B is valid, $c^* \leq B$. Exhaustion of every candidate with cost $< B$ establishes that no valid proof has lower cost, so $c^* \geq B$. Therefore $c^* = B$. $\square$

This elementary theorem is the bridge between the fast and exhaustive wings. A fast engine does not need to prove optimality. It only needs to lower B .

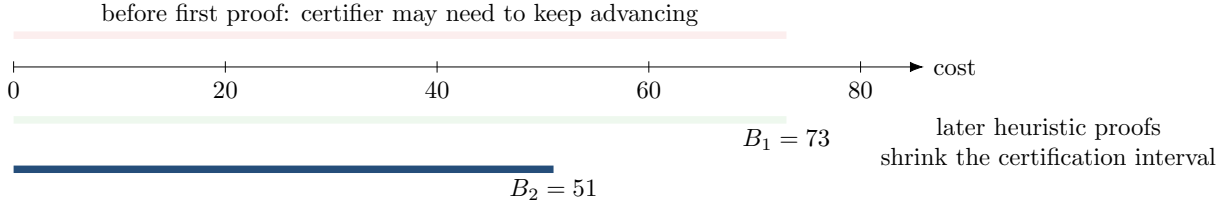


Figure 4: An incumbent proof turns the unbounded search into a bounded optimality question below B . Better verified proofs shrink that interval further.

7.1 Two success states

DATA 2.0 should report at least two positive states:

$$\text{VERIFIED PROOF FOUND} \iff \exists p V(p, \ulcorner \varphi \urcorner) = 1.$$

MINIMUM-COST PROOF CERTIFIED

means that a verified incumbent exists and every cheaper layer has been exhausted. A timeout before either condition is met must be reported operationally as NOT FOUND WITHIN BUDGET; it does not license “false,” “independent,” or “unprovable.”

8 SAT/UNSAT as an optional refutation method

The native Proof-Horizon path does not require first-order logic. Nevertheless, when a sound translation to a suitable first-order problem is available, SAT can serve as an auxiliary engine.

Given premises Γ and target φ , the usual refutation transformation is

$$\Gamma \models \varphi \iff \Gamma \cup \{\neg \varphi\} \text{ is unsatisfiable.} \quad (3)$$

For first-order clause sets, instantiation-based methods can expose finite propositional approximations. Let

$$G_0 \subseteq G_1 \subseteq G_2 \subseteq \dots$$

be cumulative finite ground clause sets generated by an explicit fair policy.

Proposition 8.1 (Persistence of cumulative UNSAT). *If $G_n \subseteq G_k$ for $k > n$ and G_n is propositionally UNSAT, then G_k is UNSAT.*

Proof. Any satisfying assignment for G_k would satisfy every clause in the subset G_n , contradicting UNSAT of G_n . $\square$

Therefore one may define the first UNSAT grounding stage

$$N^* = \min\{n : G_n \text{ is UNSAT}\},$$

when it exists. This is a useful *hitting-depth statistic*, but it is not the same quantity as native proof length or minimum proof cost.

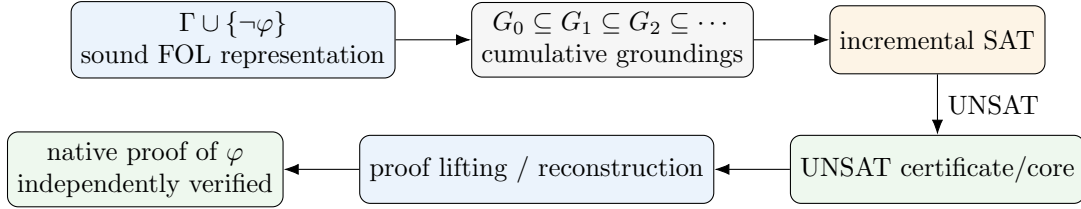


Figure 5: Optional first-order-to-SAT method. A SAT result at finite grounding depth does not refute the target. A certified UNSAT result can guide native proof reconstruction, but the native verifier remains the final authority.

The architecture resembles established instantiation-based theorem proving in its modular use of propositional reasoning. Its special role inside DATA 2.0 is as an optional discovery method feeding a native incumbent, not as a replacement for the Proof-Horizon guarantee.

9 The τ -polynomial method

The author’s earlier SAT work [2] defines, for a Boolean formula ϕ , a polynomial translation $\tau(\phi)$ over $\mathbb{F}_2$ by

$$\begin{aligned}
 \tau(p_i) &= x_i, \\
 \tau(\neg\alpha) &= 1 + \tau(\alpha), \\
 \tau(\alpha \wedge \beta) &= \tau(\alpha)\tau(\beta), \\
 \tau(\alpha \vee \beta) &= 1 + (1 + \tau(\alpha))(1 + \tau(\beta)).
 \end{aligned}$$

Together with the Boolean field equations $x_i^2 + x_i = 0$, this converts finite propositional satisfiability into an algebraic ideal-membership/variety question.

For DATA 2.0 this machinery should initially be treated conservatively. It is an optional representation of a finite propositional subproblem, not a claim that Gröbner-basis computation gives a polynomial-time SAT algorithm. The safe architectural use is:

$$\begin{aligned}
 G_n &\longrightarrow \text{propositional formula} \longrightarrow \tau(G_n) \\
 &\longrightarrow \text{algebraic analysis} \\
 &\longrightarrow \text{guidance or separately checkable certificate.}
 \end{aligned}$$

The method may be useful when algebraic structure is unusually favorable, when a polynomial-calculus certificate is natural, or when algebraic features provide better heuristic guidance than clause-level features. Whether that happens often enough to justify the cost is an empirical question.

10 Proof horizons, hitting quantities, and progress measures

DATA 2.0 naturally produces several quantities that should not be conflated.

Definition 10.1 (Proof-Horizon cost). *For a theorem target φ , define*

$$H_c(\varphi) = c^*(\varphi),$$

the minimum native proof cost under the selected effectively proper cost function.

Definition 10.2 (First-proof discovery cost). *If a heuristic wing first discovers a verified proof p at wall-clock time t_1 , search expansion count e_1 , and proof cost B_1 , record*

$$D(\varphi) = (t_1, e_1, B_1).$$

This is a property of the implementation and search policy, not only of the theorem.

Definition 10.3 (SAT hitting depth). *When the optional cumulative grounding sequence is used, N^* is the first index for which G_{N^*} is UNSAT.*

These quantities answer different questions:

Quantity	Question answered
c^*	How cheap can a valid native proof be under the chosen formal presentation and cost?
B_1	How cheap was the first proof actually discovered by this heuristic run?
t_1, e_1	How much search effort was spent before a proof was found?
N^*	At what finite grounding stage did the optional SAT method first expose contradiction?

The separation matters because endpoint quality and search expense can behave oppositely. A 12-step proof may be extremely difficult to discover, while a 60-step proof may be found immediately. DATA 2.0 should therefore log proof cost and search cost separately.

11 Proof-state graphs and the finite-branching shortest-step case

Let $\mathcal{G} = (\mathcal{S}, \mathcal{E})$ be a directed graph whose vertices are proof states and whose labeled edges are legal inference applications. An initial vertex S_0 contains the permitted starting information. Target vertices are those whose state contains or derives φ .

Theorem 11.1 (Fewest-step path under effective finite branching). *Suppose every reachable state has a finite, effectively listable set of outgoing legal inference edges and every edge has unit cost. Breadth-first search from S_0 halts exactly when a target state is reachable, and the first target reached has minimum graph distance from S_0 . The predecessor map reconstructs a proof with the minimum number of edges.*

Proof. Breadth-first search enumerates states in nondecreasing graph distance. Effective finite branching makes each finite depth layer finite and computable. Therefore no target at greater depth is dequeued before all targets at smaller depth have been ruled out. Backtracking predecessor labels reconstructs a path of that minimum depth. $\square$

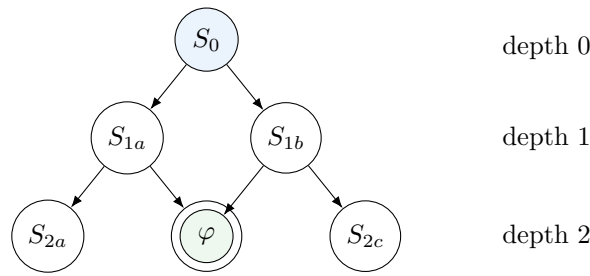


Figure 6: Finite-branching unit-cost proof graph. The first target found by BFS is a fewest-edge proof target, and predecessor labels recover the vertical proof.

The figure intentionally does not imply that realistic `set.mm` search is automatically small or uniformly finite-branching under every representation. That property is an engineering and formalization obligation of any literal shortest-step mode.

12 Generalization beyond `set.mm` and beyond first-order logic

The native Proof-Horizon layer is more general than the first-order SAT method. Its assumptions concern effective syntax, proof verification, and proof cost, not a particular logic. Accordingly, DATA 2.0 has a layered portability model.

1. **Native core.** Implement parser, proof-object encoder, verifier adapter, legal move generator, cost function, and certificate reconstructor for the target formal system.
2. **Horizon layer.** Enumerate native proof candidates in nondecreasing effectively proper cost; or use finite-branching BFS where the stronger hypotheses are established.
3. **Heuristic layer.** Attach any search policy that proposes native candidates or verified lemmas.
4. **Optional translators.** Add first-order/SAT, SMT, algebraic, or other auxiliary methods only when a sound translation and a route back to native certification are available.

Thus ZFC is an important instance, not the definition of the architecture. A frozen Metamath database is attractive experimentally because the native proof language and independent verifier are explicit and the benchmark can be contamination-controlled. Other proof assistants or calculi could be supported with different adapters.

13 Comparison with proof-search and certificate literature

The purpose of this section is methodological: locate the DATA 2.0 components relative to established ideas without claiming novelty for the components themselves.

Source / system	Established idea	Relation to DATA 2.0
Cook and Reckhow (1979)	Formal proof systems and the study of minimum proof length/relative efficiency.	Reinforces that proof length is representation-dependent and is a central proof-complexity quantity rather than a universal absolute.
Megill and Wheeler, <i>Metamath</i>	Small, transparent proof language and mechanically checkable proof databases including <code>set.mm</code> .	Motivates native-certificate output and an external verifier as the final authority.
Robinson (1965)	Resolution as a machine-oriented refutation principle for first-order logic.	Historical context for refutation-based ATP; DATA 2.0 does not replace native proof search with resolution.
Korovin, iProver (2008)	Complete instantiation-based FOL proving combined modularly with propositional SAT reasoning.	Close prior architecture for the optional grounding/SAT method; demonstrates that FOL-to-propositional modularity is established practice.
Heule, Wetzler, Hunt / DRAT	Solver-generated clausal proofs checked by a separate proof checker.	Supports the general separation of discovery from certificate checking used throughout DATA 2.0.
Suda and modern guided ATP work	Learned guidance improves clause/proof-search policy while the calculus supplies correctness.	Parallel motivation for a fast heuristic wing whose policy is distinct from verification.
Tenneson, <i>Universal Proof-Horizon Operator</i> (2026)	Partial computable recovery of canonical minimum-cost proofs under effective properness; finite-branching condition for fewest-step BFS.	Supplies the explicit exhaustive certification backbone of DATA 2.0.

Table 1: Prior-art boundary: most individual ingredients are established. The present proposal is a particular hybridization around a Proof-Horizon certifier and a shared incumbent.

The strongest architectural caution is that “heuristic search plus independent verifier” is not new. The potentially distinctive research question is whether an exhaustive Proof-Horizon wing can be made practically useful as an *optimality certifier* when paired with aggressive upper-bound discovery, rather than being used as the primary way to find proofs from scratch.

14 Implementation plan: a first executable DATA 2.0

A first implementation should be deliberately narrower than the full research program. The baseline target is a known theorem in a frozen `set.mm` snapshot.

14.1 Phase 0 modules

1. **Database loader/parser.** Read the frozen formal library and construct a normalized internal representation.
2. **Target masker.** Remove the stored proof of φ and prevent direct target leakage under the benchmark rule.
3. **Verifier adapter.** Provide a simple API that independently checks candidate native proofs.
4. **Legal move generator.** Enumerate admissible assertion applications, substitutions, hypotheses, and side conditions under the chosen search representation.

5. **Proof-state store.** Hash states, suppress duplicates where sound, and record costs.
6. **Predecessor map.** Record enough information to reconstruct $\langle \psi_1, \dots, \psi_n \rangle$ and the native certificate.
7. **Horizon engine.** Implement iterative cost-layer enumeration or, on a deliberately finite-branching restricted benchmark, breadth-first search.
8. **Fast engine.** Begin with a transparent best-first or beam policy; later add Data-like portfolio agents and learned ranking.
9. **Incumbent manager.** Store the best verified proof, publish B , and prune candidates whose rigorous lower-bound cost is already $\geq B$.
10. **Logger/checkpointer.** Save expansions, costs, branch factors, agent contributions, verifier calls, wall time, memory, and deterministic breadcrumbs.

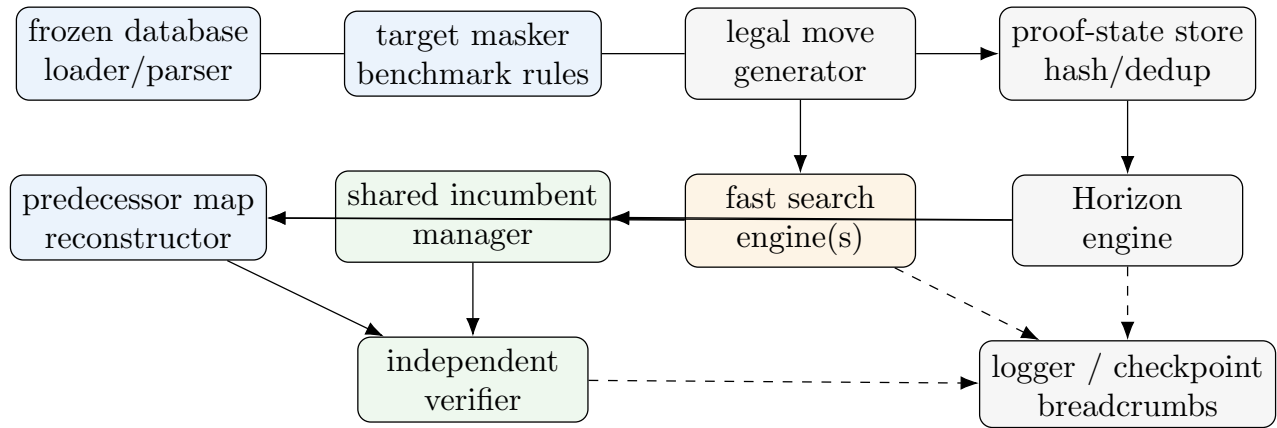


Figure 7: Phase 0 implementation modules. The shortest route to an experiment is native and modular; SAT, τ , fuzzy, flow, and semigroup investigations can be attached later without blocking the baseline.

14.2 Reference pseudocode

Listing 1: DATA 2.0 known-theorem control loop.

```

input: frozen formal system T, allowed library Gamma, target phi,
       verifier V, effectively proper cost c, budget R

initialize fast_wing
initialize horizon_wing
incumbent = NONE
B = infinity

while budget remains:
    for candidate in fast_wing.step():
        if V(candidate, phi):
            if c(candidate) < B:
                incumbent = candidate
                B = c(candidate)
                publish_bound(B)

```

```

for candidate in horizon_wing.next_candidates_below(B):
    if V(candidate, phi):
        incumbent = canonical_better(candidate, incumbent)
        B = c(incumbent)
        publish_bound(B)

if incumbent exists and horizon_wing.exhausted_all_costs_below(B):
    return MINIMUM_COST_VERIFIED_PROOF(incumbent)

if incumbent exists and user_policy_allows_early_return:
    report VERIFIED_PROOF_FOUND(incumbent)

return NOT_FOUND_WITHIN_BUDGET

```

A practical implementation may run the two wings in separate processes and communicate only through a verified incumbent store, a bound channel, and a shared immutable benchmark description. That separation makes it easier to debug whether an apparent gain came from heuristic discovery, proof verification, or exhaustive certification.

15 Benchmark illustrations, not performance claims

The following examples illustrate what should be measured; they are not empirical results.

Suppose a heuristic agent finds a valid proof at cost 73. The system immediately knows

$$c^* \leq 73.$$

If a second agent later finds cost 51, then

$$c^* \leq 51.$$

The Horizon certifier can now ignore every candidate whose rigorous lower bound is at least 51 and needs only to finish the cheaper region. If it exhausts all candidates below 51, the cost-51 proof is optimal.

Now compare two hypothetical theorems:

Target	first proof cost	first-proof expansions	certified optimum	certification expansions
A	12	2,000,000	12	5,000,000
B	60	8,000	41	70,000,000

These numbers are deliberately invented. They show why proof quality and search cost must be recorded separately. Target A has a short proof that is hard to discover. Target B yields a long proof quickly, but proving that a much shorter optimum exists may require extensive certification.

A fair comparison with Data-ATP, Predator, or another ATP should therefore record at least: success/-failure within budget, time to first verified proof, expansions to first proof, memory, first proof cost, best proof cost found by deadline, time to minimum-cost certification if achieved, and the contribution of each search engine to incumbent improvements.

16 Open mathematical and engineering questions

This first paper deliberately ends with unresolved questions, paralleling the open-question structure of the multi-dose concentration manuscript.

16.1 Question 1: which native proof cost should be primary?

Encoded bit length, symbol count, proof-file bytes, expanded proof steps, and weighted inference cost answer different questions. Which measure best supports both effective properness and meaningful comparison with human- or ATP-produced proofs?

16.2 Question 2: when does `set.mm` admit a useful finite-branching shortest-step graph?

A frozen finite assertion library does not by itself prove that every search state has only finitely many relevant substitutions under every representation. The exact native search state and admissible move generator must be specified before a literal fewest-step BFS theorem is invoked.

16.3 Question 3: how much does an incumbent accelerate certification in practice?

The hybrid architecture is useful only if good upper bounds significantly reduce the exhaustive region. This should be measured across theorem classes rather than assumed.

16.4 Question 4: how should heuristic lemmas interact with the Horizon wing?

A heuristic wing may discover useful intermediate lemmas. If those lemmas are admitted as new primitives, the proof presentation changes and so does the meaning of minimum cost. A clean benchmark may permit lemma discovery only when the final native proof is fully expanded back into the original allowed library.

16.5 Question 5: when is the SAT method useful?

For suitable first-order problems, cumulative grounding and SAT can expose contradiction efficiently. For native Metamath goals, translation and proof lifting may dominate the benefit. The method should earn its complexity experimentally.

16.6 Question 6: when is the τ -algebraic method useful?

The polynomial translation is exact at the Boolean level, but Gröbner-basis complexity can be severe. The relevant question is not whether the translation exists, but whether particular proof-search subproblems have exploitable algebraic structure.

16.7 Question 7: can proof-state dynamics supply useful progress coordinates?

The earlier working notes considered semigroups, flows, logarithmic progress, and continuous relaxations. None is required by DATA 2.0. A later paper can ask whether a state quantity $Q(S)$ or coordinate $\Psi(S)$ predicts proof proximity well enough to improve the fast wing without corrupting the independent certification layer.

16.8 Question 8: how should known-theorem mode become conjecture mode?

The present architecture assumes the target is known to be a theorem for benchmarking. A future ALD version can dovetail a proof search for C , a proof search for $\neg C$, and a genuinely separate model/independence arm. Prolonged failure of both proof searches must remain UNKNOWN, not independence.

17 Conclusion

DATA 2.0 begins from a simple separation of responsibilities. The target is φ . The desired certificate is a finite native proof

$$\Pi = \langle \psi_1, \dots, \psi_n \rangle, \quad \psi_n = \varphi.$$

A fast wing tries to find such a proof quickly. A Proof-Horizon wing searches completely in increasing effectively proper cost and, when the stronger finite-branching hypotheses hold, can specialize to breadth-first fewest-step search. The independent verifier decides whether a candidate counts.

The key hybrid move is the shared incumbent. Once any fast engine finds a verified proof of cost B , the exhaustive question changes from “search forever until some proof is found” to “does any proof cheaper than B exist?” If every cheaper candidate is exhausted, the incumbent is certified minimum-cost. Better heuristic proofs tighten the bound and can reduce the remaining certification region. This gives the Universal Proof-Horizon paper a concrete computational role: it supplies the exhaustive optimality backbone, while Data-like agents supply navigation.

SAT/UNSAT grounding, the author’s τ polynomial translation, learned guidance, semigroup ideas, fuzzy relaxations, and other experimental machinery can be added as optional methods. None is permitted to redefine correctness. The native proof and external verifier remain the final certificate path.

The first implementation should therefore be smaller than the full vision: freeze `set.mm`, mask a known theorem’s stored proof, define a defensible native cost, implement a simple Horizon engine and a transparent fast engine, share verified incumbents, reconstruct the proof vertically, verify it externally, and measure what happens. DATA 2.0 may be faster than Data-ATP on some theorem classes and slower on others. That comparison is not a weakness of the proposal; it is the experiment the architecture is designed to make possible.

Acknowledgements, AI-use statement, and provenance

This manuscript develops ideas from Brian Tenneson’s *Universal Proof-Horizon Operator* and from his broader automated-theorem-proving research. The Proof-Horizon paper supplies the formal minimum-cost recovery framework and explicitly distinguishes effectively proper proof cost from literal line-count minimality. The DATA 2.0 hybrid architecture was developed by connecting that framework to the author’s Data-ATP/Predator program, known-theorem benchmarking, native Metamath verification, SAT/UNSAT methods, and the author’s earlier τ Boolean-polynomial translation. OpenAI ChatGPT was used as an interactive collaborator for mathematical organization, literature retrieval and comparison, architecture criticism, diagrams, and LaTeX preparation. The author directed the research questions and final architectural choices. All claims of proof correctness are intended to be independently checkable; model output is not treated as a proof authority.

References

- [1] B. Tenneson, *The Universal Proof-Horizon Operator: From MIU and the n th Well-Formed Formula to ZFC, Predator, and Automated Logical Deciders*, Version 1.0, August 3, 2026. [GitHub copy](#).
- [2] B. Tenneson, *Solving SAT in Polynomial Time?*, working manuscript, March 22, 2025. The present paper uses only the exact Boolean-to-polynomial translation τ as an optional representation and makes no polynomial-time SAT claim. [GitHub copy](#).
- [3] N. D. Megill and D. A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, 2nd ed., 2019. Metamath project documentation, <https://us.metamath.org/downloads/metamath.pdf>.
- [4] S. A. Cook and R. A. Reckhow, “The Relative Efficiency of Propositional Proof Systems,” *Journal of Symbolic Logic* 44(1), 36–50, 1979. DOI: [10.2307/2273702](#).
- [5] J. A. Robinson, “A Machine-Oriented Logic Based on the Resolution Principle,” *Journal of the ACM* 12(1), 23–41, 1965. DOI: [10.1145/321250.321253](#).
- [6] K. Korovin, “iProver – An Instantiation-Based Theorem Prover for First-Order Logic (System Description),” in *Automated Reasoning, IJCAR 2008*, LNCS 5195, 292–298, 2008. DOI: [10.1007/978-3-540-71070-7_24](#).
- [7] M. J. H. Heule, N. Wetzler, and W. A. Hunt Jr., “DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs,” in *Theory and Applications of Satisfiability Testing – SAT 2014*, LNCS 8561, 422–429, 2014. DOI: [10.1007/978-3-319-09284-3_31](#).
- [8] M. Suda, “Vampire With a Brain Is a Good ITP Hammer,” arXiv:2102.03529, 2021.
- [9] A. Teucke and C. Weidenbach, “First-Order Logic Theorem Proving and Model Building via Approximation and Instantiation,” arXiv:1503.02971, 2015.